

计算机系统基础 1

PPT 补充考点专项模拟题（第 5 套）

磁盘访问与 I/O · 嵌套/多层数组 · 可变大小数组 · 联合体 · 字节序

总分 / 建议时间	100 分 / 120 分钟
出题依据	第 10 章“程序的机器级表示：数据”与第 13 章“存储器层次结构”课件中，往年卷较少直接考查的内容。
默认环境	x86-64 Linux：char=1B，short=2B，int=4B，long=8B，指针=8B；系统采用小端字节序。

使用说明

- 本卷刻意避开往年题中已反复出现的补码、基础 Cache 组索引、常规结构体对齐和强弱符号题，改为检验课件中同样可能出题、但往年卷覆盖较少的知识点。
- 做题时请写清关键公式、地址推导过程和单位。磁盘时间问题可保留 3 位小数；涉及联合体时，应写清“同一片内存的不同解释”，不要误写成数值类型转换。
- 答案部分不仅给出结论，还标明常见误区，建议先独立完成后再对照。

本卷覆盖矩阵

专题	主要题号	对应课件关键词
磁盘服务时间与 I/O	选择 1-5；填空 1-3；综合 1-2	寻道、平均旋转延迟、传输时间、逻辑块、DMA、中断、SSD 页/块
嵌套数组与多层数组	选择 6-9；填空 4-5；综合 3-4	行优先、A[i][j] 地址、数组行、int (*)[C] 与 int *[R]、两次访存
联合体与字节序	选择 10-12；填空 6-8；综合 5-6	最大成员分配、内存覆盖、float/unsigned 位级重解释、小端字节序

试题部分

一、单项选择题（每题 2 分，共 24 分）

1. (2 分) 对于机械硬盘读取一个目标扇区，下面最符合课件近似模型的平均服务时间公式是（ ）。
 - A. $T_{\text{access}} = T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}}$
 - B. $T_{\text{access}} = T_{\text{avg seek}} \times T_{\text{avg rotation}} \times T_{\text{avg transfer}}$
 - C. $T_{\text{access}} = T_{\text{avg transfer}} - T_{\text{avg seek}} - T_{\text{avg rotation}}$
 - D. T_{access} 只由磁盘 RPM 决定
2. (2 分) 一块转速为 7200 RPM 的磁盘，其平均旋转延迟最接近（ ）。
 - A. 8.33 ms
 - B. 4.17 ms
 - C. 2.08 ms
 - D. 0.14 ms
3. (2 分) 在其他条件相同的情况下，下面哪项通常不会直接改变“读一个给定扇区的传输时间” $T_{\text{avg transfer}}$ ？（ ）
 - A. 每分钟转数 RPM
 - B. 每磁道平均扇区数
 - C. 目标扇区的数据量
 - D. 读写头到目标柱面的移动距离
4. (2 分) CPU 发起一次磁盘读操作，关于 DMA 和磁盘控制器的描述正确的是（ ）。
 - A. CPU 必须逐字节从磁盘端口读取数据并写入主存
 - B. 控制器读取扇区后可通过 DMA 将数据直接传送到主存，完成时再中断 CPU
 - C. DMA 只能在 CPU 停机时工作，且不能访问主存
 - D. 控制器仅负责转动磁盘，不能维护逻辑块到物理扇区的映射
5. (2 分) 关于 SSD 的说法，正确的是（ ）。
 - A. SSD 读取、写入与擦除的最小单位都相同，通常都是单个字节
 - B. SSD 没有任何写入限制，因此比 DRAM 更适合作为主存
 - C. SSD 通常以页为单位读写，以块为单位擦除，擦除操作相对较慢
 - D. SSD 的随机读写必须经历寻道时间和旋转延迟
6. (2 分) 设 `typedef int zip_dig[5]; zip_dig pgh[4];` 下列描述正确的是（ ）。
 - A. pgh 是 4 个 int 指针组成的数组
 - B. pgh 等价于 `int pgh[4][5]`，所有 20 个 int 连续按行优先存放
 - C. `pgh[index]` 的类型是 int，大小为 4B
 - D. pgh 各行一定分散在不同内存页，不能连续
7. (2 分) 二维数组 `T A[R][C]` 中，若 T 类型元素大小为 K 字节，A 的首地址为 A，则 `A[i][j]` 的地址为（ ）。
 - A. $A + (i \times R + j) \times K$

- B. $A + (j \times C + i) \times K$
- C. $A + (i \times C + j) \times K$
- D. $A + (i + j) \times R \times C \times K$

8. (2分) 给定 `int *univ[3]`, 表达式 `univ[index][dig]` 的一次取值通常需要进行两次内存读取, 其原因是 ()。

- A. 先读 `univ[index]` 中保存的行数组指针, 再按该指针读取第 `dig` 个 `int`
- B. 先读取数组长度, 再读取元素
- C. 先对 `index` 做符号扩展, 再读取元素
- D. 因为 `int` 必须跨两个 `cache` 块存放

9. (2分) 关于 `int A[4][5]` 与 `int *B[4]` 的说法, 正确的是 ()。

- A. 二者在内存布局和 `A[i][j]/B[i][j]` 的寻址步骤上完全相同
- B. `A` 的所有元素保证连续; `B` 仅保证 4 个指针连续, 各行数据是否连续取决于各指针值
- C. `B` 的每个元素都占 4B, 因此 `B` 总大小是 16B
- D. `A[i]` 的类型是 `int *`, 且一定由独立 `malloc` 得到

10. (2分) 关于 `union U { char c; int i[2]; double d; }` 的空间分配, 正确的是 ()。

- A. `union` 的总大小等于所有成员大小之和
- B. `union` 的成员在内存中按声明顺序依次排开
- C. `union` 按最大成员的大小并满足最大对齐要求分配空间, 成员起始地址重合
- D. `union` 每次可同时保存三个彼此独立的成员值

11. (2分) 以下 `bit2float` 的实现与 `(float)u` 的本质区别是 ()。

`float bit2float(unsigned u) { bit_float_t arg; arg.u = u; return arg.f; }`

- A. 二者都是把无符号整数的数值转换为浮点数数值
- B. 前者通过 `union` 将同一组位重新解释为 `float`, 后者是数值转换
- C. 前者一定对 `u` 做符号扩展, 后者不做任何变化
- D. 前者会改变 `u` 在内存中的位模式

12. (2分) 在 x86-64 小端机器上, `union { unsigned char c[4]; unsigned short s[2]; } u;` 若依次令 `u.c[0..3] = {0xF0, 0xF1, 0xF2, 0xF3}`, 则 `u.s[0]` 的值为 ()。

- A. 0xF0F1
- B. 0xF1F0
- C. 0xF2F3
- D. 0xF3F2

二、填空题 (每题 2 分, 共 16 分)

1. (2分) 某磁盘转速为 10000 RPM, 平均旋转延迟为 _____ ms。

2. (2分) 某磁盘转速为 7200 RPM, 平均每磁道有 400 个扇区, 读取一个扇区的平均传输时间约为 _____ ms。

3. (2分) 某磁盘平均寻道时间为 6 ms，转速为 10000 RPM，每磁道 1000 个扇区。读取一个扇区的平均访问时间约为 _____ ms。
4. (2分) int A[6][8] 的首地址为 0x1000。则 &A[3][5] = _____ (十六进制)。
5. (2分) int *row[4] 的首地址为 0x2000。假设指针大小为 8B，则存放 row[3] 指针值的内存地址为 _____ (十六进制)。
6. (2分) 在 x86-64 平台上，union U { char c; int i; double d; }; 的 sizeof(U) 为 _____ 字节。
7. (2分) typedef union { float f; unsigned u; } bit_float_t; 若 arg.u = 0x3F800000，则 arg.f 的数值为 _____。
8. (2分) 声明 int (*p)[5] 中，p 的含义是 “_____”；而 int *q[5] 中，q 的含义是 “_____”。

三、综合题 (共 60 分)

1. (12分) 磁盘访问时间计算。某机械硬盘转速为 7200 RPM，平均寻道时间为 8 ms，平均每磁道有 400 个扇区。假设读取一个扇区，且不考虑控制器额外开销。

(1) (3分) 求平均旋转延迟 Tavg rotation。

(2) (3分) 求读取一个扇区的平均传输时间 Tavg transfer。

(3) (3分) 求平均访问时间 Taccess。结果保留两位小数 ms。

(4) (3分) 若从同一磁道连续读取 8 个相邻扇区，并且只发生一次寻道和一次旋转等待，平均服务时间约为多少？说明这体现了什么结论。

2. (8分) 磁盘 I/O、逻辑块与 DMA。请用 4 个步骤概述 CPU 发起“读取一个逻辑磁盘块到主存缓冲区”的过程。要求说明：谁发起请求、控制器做什么、DMA 做什么、何时通知 CPU。

3. (14分) 嵌套数组与多层数组寻址。

```
typedef int zip_dig[5];
zip_dig pgh[4];           // 等价于 int pgh[4][5]
int *univ[3];
```

(1) (5分) pgh 的起始地址为 0x1000。求 pgh[3] 的起始地址与 &pgh[3][4] 的地址。写出计算式。

(2) (5分) univ 的起始地址为 0x2000，且 univ[0]=0x3000，univ[1]=0x4000，univ[2]=0x5000。求 &univ[1][4] 的地址，并说明取 univ[1][4] 的值通常要进行哪两次内存读取。

(3) (4分) 比较 pgh 与 univ 的内存连续性、行长度约束和寻址特点。

4. (10分) 可变大小数组 (VLA) 与访问局部性。设函数参数为 long A[n][n]，元素大小为 8B，A 的首地址为 base。

(1) (3分) 写出 A[i][j] 的地址表达式。

(2) (4分) 分析下面代码的数据访问局部性。指出问题并给出一种更好的循环组织方式 (无需写完整代码)。

```
long sum_col(size_t n, long A[n][n], size_t j) {
    long s = 0;
    for (size_t i = 0; i < n; i++)
        s += A[i][j];
    return s;
}
```

(3) (3分) 说明为什么 `int **P` 不能直接当作与 `int A[n][n]` 完全等价的二维数组参数。

5. (10分) 联合体、覆盖存储与小端字节序。

```
union U {
    unsigned char c[8];
    unsigned short s[4];
    unsigned int i[2];
    unsigned long l;
} u;

for (int k = 0; k < 8; k++) u.c[k] = 0xF0 + k;
```

(1) (6分) 在 x86-64 小端机器上，分别写出 `u.s[0]`、`u.s[3]`、`u.i[0]`、`u.i[1]` 与 `u.l` 的十六进制值。

(2) (4分) 解释这些成员为什么能够“读出不同类型的值”，并指出 `union` 的空间分配规则。

6. (6分) 联合体的位级重解释。

```
typedef union { float f; unsigned u; } bit_float_t;

float bit2float(unsigned u) {
    bit_float_t arg;
    arg.u = u;
    return arg.f;
}
```

(1) (2分) `bit2float(0xC0400000)` 的返回值是多少？

(2) (4分) 说明为什么其结果不同于 `(float)0xC0400000u`。

参考答案与详细解析

一、单项选择题

1. (2分) 答案: A

解析: 课件给出的近似模型为 $T_{\text{access}} = T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}}$ 。三项分别对应读写头移动到目标柱面、等待目标扇区旋转到读写头下方、读取目标扇区数据。

2. (2分) 答案: B

解析: 7200 RPM 表示每分钟 7200 圈。一圈时间为 $60/7200 \text{ s} = 8.333 \text{ ms}$ 。平均只需等待半圈, 因此平均旋转延迟约为 4.167 ms。

3. (2分) 答案: D

解析: 寻道距离影响的是寻道时间, 不是“扇区数据从盘面经过读写头”的传输时间。传输时间主要与转速、每磁道扇区数和读取数据量有关。

4. (2分) 答案: B

解析: CPU 通过向控制器相关端口/寄存器写入命令和目标主存地址来启动读操作; 控制器完成物理读后可通过 DMA 直接将数据搬到主存, 完成时向 CPU 发中断。

5. (2分) 答案: C

解析: 课件强调 SSD 通常以页为单位读写, 以块为单位擦除; 块擦除较慢。SSD 不存在机械硬盘的寻道和旋转延迟, 但擦除、写放大和寿命仍是重要问题。

6. (2分) 答案: B

解析: `zip_dig` 是“含 5 个 int 的数组类型”, 所以 `pgh` 是 4 个 `zip_dig` 构成的数组, 即 `int pgh[4][5]`。二维嵌套数组整体连续, 采用行优先排列。

7. (2分) 答案: C

解析: 二维嵌套数组按行优先。跳过 i 行需要 $i \times C$ 个元素, 再跳过当前行前 j 个元素, 因此偏移元素数为 $i \times C + j$, 再乘元素大小 K 。

8. (2分) 答案: A

解析: `univ` 是“指针数组”。先需要读取 `univ[index]` 中的指针值, 得到行数组地址; 再在该地址处读取第 `dig` 个 int, 因此通常有两次内存读取。

9. (2分) 答案: B

解析: A 是真正的二维嵌套数组, 所有 20 个 int 连续。B 本身只是 4 个指针连续, 各行指向的数组可以分散、长度也可以不同; `B[i][j]` 需要先间接取行指针。

10. (2分) 答案: C

解析: union 的所有成员从同一起始地址解释同一片存储空间。其总大小至少能容纳最大成员, 并满足最大对齐要求; 它不是把每个成员依次独立保存。

11. (2分) 答案: B

解析：union 写 arg.u 再读 arg.f，是将完全相同的 32 位模式用 float 规则解释；(float)u 是数值转换，先把 u 作为整数值，再求相应浮点数的数值编码。

12. (2 分) 答案：B

解析：小端中低地址保存低有效字节。c[0]=F0 位于低地址、c[1]=F1 位于高地址，16 位 s[0] 的数值为 0xF1F0。

二、填空题

1. (2 分) 答案：3 ms

解析：一圈时间 = $60/10000 \text{ s} = 6 \text{ ms}$ ；平均旋转延迟取半圈，因此为 3 ms。

2. (2 分) 答案：约 0.0208 ms

解析：一圈 $60/7200 \text{ s} = 8.333 \text{ ms}$ ；平均每磁道 400 个扇区，所以读取一个扇区约为 $8.333/400 \text{ ms} = 0.020833 \text{ ms}$ 。

3. (2 分) 答案：约 9.006 ms

解析：Tseek=6 ms；题中转速为 10000 RPM，所以半圈等待=3 ms；传输时间=一圈 6 ms/1000=0.006 ms。总计 $6+3+0.006=9.006 \text{ ms}$ 。

4. (2 分) 答案：0x1074

解析：行优先：元素序号 = $3 \times 8 + 5 = 29$ ；字节偏移 = $29 \times 4 = 116 = 0x74$ ；地址 = $0x1000 + 0x74 = 0x1074$ 。

5. (2 分) 答案：0x2018

解析：row 是含 4 个 8B 指针的数组。row[3] 指针自身存放在 row 首地址 + $3 \times 8 = 0x2000 + 0x18 = 0x2018$ 。

6. (2 分) 答案：8

解析：最大成员 double 为 8B，最大对齐要求通常也是 8B，因此 union 至少占 8B。其余成员不额外叠加空间。

7. (2 分) 答案：1.0

解析：0x3F800000 是 IEEE 754 单精度 float 中 1.0 的位级编码。union 将这一编码直接按 float 解释。

8. (2 分) 答案：指向“含 5 个 int 的数组”的指针；含 5 个“指向 int 的指针”的数组

解析：括号位置决定结合方式：(*p)[5] 先解引用 p，再得到一个 5 元素 int 数组；q[5] 先形成 5 个元素的数组，每个元素是 int 指针。

三、综合题

1. (12 分) 答案：(1) 4.17 ms；(2) 0.0208 ms；(3) 约 12.19 ms；(4) 约 12.33 ms，连续读的额外成本主要只是额外传输时间。

解析：(1) 7200 RPM 时一圈时间 = $60/7200 \text{ s} = 8.333 \text{ ms}$ ，平均旋转延迟 = 半圈 = 4.167 ms。

(2) 单扇区平均传输时间 = $8.333/400 = 0.020833 \text{ ms}$ 。

(3) Taccess = $8 + 4.167 + 0.020833 = 12.187833 \text{ ms}$ ，约为 12.19 ms。

(4) 连续读 8 个相邻扇区，寻道和旋转等待仍只发生一次；传输时间变为 $8 \times 0.020833 = 0.166664 \text{ ms}$ ，所以约为 $8 + 4.167 + 0.167 = 12.334 \text{ ms}$ ，即约 12.33 ms。课件结论是：定位到第一个扇区的第一个比特最“昂贵”，随后连续数据的边际读取成本很低。

2. (8分) 答案: CPU 发起命令 -> 控制器定位和读取 -> DMA 搬运 -> 中断通知。

解析: 可按以下四步组织答案:

- 1) CPU 向磁盘控制器关联的端口/寄存器写入读命令、逻辑块号以及主存缓冲区地址。
- 2) 控制器依据逻辑块到物理扇区的映射, 完成寻道、旋转等待并读取对应扇区。
- 3) 控制器启动 DMA, 将读取到的数据直接写入指定主存缓冲区, CPU 不需要逐字节参与拷贝。
- 4) DMA 传输完成后, 磁盘控制器通过中断通知 CPU, 本次 I/O 完成。

3. (14分) 答案: (1) pgh[3] 起始地址=0x103C, &pgh[3][4]=0x104C; (2) &univ[1][4]=0x4010; (3) pgh 整体连续, univ 只保证指针数组连续。

解析: (1) 每行有 5 个 int, 每个 int 4B, 所以一行占 20B=0x14。pgh[3] 地址=0x1000+3×0x14=0x103C。pgh[3][4] 再偏移 4×4=16B=0x10, 故地址为 0x104C。

(2) univ[1] 的指针值为 0x4000, univ[1][4] 地址=0x4000+4×4=0x4010。求其值时, 先从 univ 首地址 +1×8=0x2008 读出指针 0x4000; 再从 0x4010 读取 int 元素, 通常是两次内存读取。

(3) pgh 是固定列数的真正嵌套数组, 4×5 个 int 全部连续, 行长固定为 20B, 可用单一公式直接算地址。univ 是指针数组, 3 个指针连续, 但三行可位于任意地址、行长可以不一致, 必须先读取行指针再访问元素。

4. (10分) 答案: (1) base + (i×n+j)×8; (2) 当前代码按列访问, 局部性差; (3) int ** 不是同样的连续二维布局。

解析: (1) long 元素大小为 8B, 行优先下偏移元素数为 i×n+j, 故地址为 base+(i×n+j)×8。

(2) 在 sum_col 中 j 固定、i 变化, 访问 A[0][j]、A[1][j]、A[2][j].....每次跨过一整行 n 个 long, 空间局部性较差。若目标是遍历整个矩阵, 应让列下标 j 作为内层循环变量, 连续访问 A[i][0]、A[i][1].....; 若只求某一固定列, 则可使用分块/转置等方式改善整体缓存行为。

(3) int A[n][n] 表示固定列宽、整体连续的二维嵌套数组; int **P 是“指向指针的指针”, 常见实现中先访问 P[i] 再访问 P[i][j], 各行可分散且不保证连续, 因此参数类型、寻址过程和内存布局均不同。

5. (10分) 答案: u.s[0]=0xF1F0; u.s[3]=0xF7F6; u.i[0]=0xF3F2F1F0; u.i[1]=0xF7F6F5F4; u.l=0xF7F6F5F4F3F2F1F0。

解析: 循环后低地址到高地址依次是 F0 F1 F2 F3 F4 F5 F6 F7。x86-64 是小端, 低地址字节为数值的低有效字节, 因此:

- s[0] 覆盖 F0 F1, 数值为 0xF1F0; s[3] 覆盖 F6 F7, 数值为 0xF7F6。

- i[0] 覆盖 F0 F1 F2 F3, 数值为 0xF3F2F1F0; i[1] 为 0xF7F6F5F4。

- l 覆盖全部 8 字节, 数值为 0xF7F6F5F4F3F2F1F0。

union 的成员共享同一起始地址、覆盖同一片内存; 其空间按最大成员大小并满足最大对齐要求分配。

6. (6分) 答案: -3.0; 因为 union 是位级重解释, 而强制类型转换是数值转换。

解析: 0xC0400000 是 IEEE 754 单精度中 -3.0 的编码。因此 bit2float 直接把同样的 32 位按 float 规则解释, 返回 -3.0。

(float)0xC0400000u 则先把 0xC0400000 当作无符号整数 3225419776, 再把这个数学数值转换为 float, 结果会是约 3.225×10^9 的正浮点数, 而不是 -3.0。这里的关键是“位模式不变的重解释”与“数值改变的类型转换”完全不同。

考后错题回填清单

错题类型	必须写出的核心规则	常见失分点
磁盘时间	$T_{access}=T_{seek}+T_{rotation}+T_{transfer}$; 平均旋转等待=半圈。	把整圈时间当成平均旋转等待；忘记把秒换算为 ms。
嵌套数组	$A[i][j]$ 地址= $base+(i \times \text{列数}+j) \times \text{元素大小}$ 。	把行数 R 错当成列数 C；忘记元素大小。
多层数组	先读行指针，再取元素；常有两次内存读取。	误以为 $\text{int} *A[R]$ 和 $\text{int} A[R][C]$ 整体连续。
联合体	成员覆盖同一片存储，按最大成员大小/对齐分配。	把 union 当 struct；把位级重解释当作数值转换。
小端	低地址保存低有效字节，按字节从低地址开始拼数值。	直接按书写顺序拼成大端数。